CS6290 HPCA — Midterm 打印笔记 Lessons: Intro → Metrics → Pipelining → Branches → Predication → ILP → Tomasulo → ROB → Memory Ordering → Compiler ILP → VLIW → Cache Review → Virtual Memory → Advanced Caches（依据课程字幕整理）

0 • 公式速查 (Formula Sheet)

主题	公式 / 结论
动态功耗	P_{dyn} = ½·C·V²·f·α（C∝面积，α=活动因子）。V↓ 则静态(leakage)功耗↑，总功耗存在最优 V。
芯片成本	cost = wafer cost / #good chips; yield = good/total；芯片越大→成本**超线性**增长（片数少+良率低）
Speedup	S = Lat_{old}/Lat_{new} = Thr_{new}/Thr_{old}; S<1 = slowdown
Iron Law	CPU time = IC × CPI × T_{clk}; 不同类指令: $\Sigma(IC_i \times CPI_i) \times T_{clk}$; T_{clk}=1/f
Amdahl	S = 1 / [(1−F) + F/S_e], **F = 执行时间占比**（不是指令占比！）
Lhadma	别把罕见情况弄太慢：1/[0.1/0.1 + 0.9/2] = 0.69（变慢）
平均性能	平均时间再求比，或对时间和 speedup 都用**几何平均**；**不能**对 speedup 做算术平均
流水线	n 条指令、k 级：k + (n−1) 周期；CPI = 1 + Σ(事件频率×代价)
分支 CPI	CPI = CPI_{ideal} + (mispred/inst) × penalty; mispred/inst = (mispred/branch)×(branch/inst)；在第 k 级 resolve → penalty = k−1
N-bit 历史	每 entry 代价 N + 2·2^N bit；可预测所有长度 **≤ N+1** 的重复模式
If-conversion	if-converted 指令数 = 两条路径 + 选择指令；branch 版本 = 平均路径 + mispred×penalty；令两者相等求临界准确率
ILP vs IPC	ILP = 理想处理器的 IPC（只看 RAW）；**IPC ≤ ILP**
AMAT	AMAT = HT + MR × MP；miss time = HT + MP；多级：MP_{L1} = HT_{L2} + MR_{L2}×MP_{L2}（MR_{L2} 用 **local**）
Cache 地址	offset = $\log_2$(block), #sets = size/(block×ways), index = $\log_2$(#sets), tag = 剩余
LRU 位数	N-way：每 set N 个 $\log_2$N bit 计数器；NMRU：每 set $\log_2$N bit；PLRU：每行 1 bit
Flat PT	#PTE = VA space / page size; size = #PTE × PTE size（与实际用量、物理内存大小无关）
多级 PT	size = (实际需要的各级表个数) × (每张表 = 2^{bits} × PTE size)
VIPT 无 aliasing	cache size ≤ associativity × page size（index+offset 位全在 page offset 内）
Prefetch 距离	pdist ≈ memory latency / 每次迭代周期，并检查预取数据到用时不会被踢出

1 • Introduction

- 体系结构 = 为**预期用途**设计合适的计算机（性能、电池、体积、重量、能效、成本、新能力如 3D/调试/安全）。造更快的晶体管**不是**体系结构。
- 要预测技术趋势：用现有技术设计，做完时已过时。**Moore's Law**：每 18–24 月同面积晶体管翻倍 → 架构师目标：速度×2、每操作能耗×½、内存容量×2。
- **Memory wall**：处理器速度、内存容量 ~2×/2年，但内存**延迟**只 ~1.1×/2年 → 差距越来越大 → 用 cache 当"台阶"。
- 速度/成本/功耗权衡：同一工艺进步可换"2× 速度同成本同功耗"或"1.1× 速度、½ 成本、½ 功耗"。极端设计（Burnium）通常最差。

功耗

- Dynamic（active）：½CV²fα。缩小 + 双核 → C 不变；降压是关键（V² 关系）。例：V×0.8 → 0.64 倍。
- Static：主要是 leakage，V 越低漏电越大 → 不能无限降压；总功耗 vs V 曲线有最小点。

例 已知 1V/2GHz = 30W。0.9V/1.8GHz：30×0.9²×0.9 = **21.9W**；1.5V/3GHz：30×1.5²×1.5 = **101.3W**。

制造成本

例 晶圆 $5000，10 缺陷：小芯片 400 片→390 好 = **$12.8**；4× 面积 96 片→86 好 = **$58.2**（≈5×而非 4×）；再 4× 面积 20 片，缺陷可能同片→损 9 片 → 11 好 = **$454**（≈8×）。

Moore 红利两种用法：(1) 同功能更小更便宜（新设备：手机、手表）；(2) 同面积更强（高性能）。1mm² vs 100mm² 芯片：成本 **>100×**。

2 • Metrics & Evaluation

- **Latency**（开始→完成）vs **Throughput**（每秒完成数）；throughput ≠ 1/latency（流水线、复制服务器）。例：2 台服务器、各 1ms/单 → latency 1ms，throughput 2000/s。
- "X 比 Y 快 N 倍"：N = Lat_Y/Lat_X。4h → 10min：speedup = 240/10 = **24**；反过来 = 0.04。
- Benchmark 类型：real applications（最具代表性、最难搭建）→ application **kernels**（真实代码的一部分：最耗时的循环）→ **synthetic**（模仿 kernel，不是真实代码，适合早期设计研究）→ **peak performance**（只适合营销）。
- 标准组织：**TPC**（数据库/Web/事务）、**EEMBC**（嵌入式：车、打印机、手机）、**SPEC**（工作站/CPU，IO 少；gcc、bwaves、lbm、perl、bzip…）。

例 X: 9,10,5s；Y: 18,7,11s。平均时间 8 vs 12 → speedup 1.5；speedup 算术平均 1.63（**错**）；几何平均：7.66, 11.15 → 1.456 = 各 speedup 几何平均。

Iron Law / Amdahl

- IC ← 算法、编译器、ISA；CPI ← ISA、处理器设计；T_{clk} ← 处理器设计、电路、工艺。复杂指令：IC↓CPI↑；短周期：T↓CPI↑ → 要平衡。
- 例：3×10^9 指令，CPI 2，3GHz → 2s。50B 指令：10B 分支×4 + 15B load×2 + 5B store×3 + 20B add×1 = 105B 周期 / 4GHz = **26.25s**。

⚠ **易错** Amdahl 的 F 必须是**原执行时间**的占比。给的是"20% 指令是分支"时，要先用 CPI 换算时间占比（或直接用 Iron Law 前后比较）。例：int 40%×1, br 20%×4→2, ld 30%×2, st 10%×3：CPI 2.1→1.7，speedup = **1.24**（不是 1/(0.8+0.1)=1.11）。

- Make the common case fast：20× on 10% → 1.105；1.6× on 80% → 1.43；∞ on 10% → 1.111 上限。
- 例（时间占比已给）：分支 20% 时间 CPI 4→3 → 1/(0.8+0.2/1.333)=1.05；频率 2→2.3GHz → 1.15（最好）；store 10% 3→2 → 1.034。
- **Diminishing returns**：每次加速同一部分后它占比变小（50/50 → 加速 2×：1.33；再 2×：只 1.2）→ 每次改进后重新评估主导部分。

3 • Pipelining

- 5 级：**F**etch → **D**ecode/读寄存器 → **A**LU → **M**EM → **W**rite reg。latency 不变，throughput ×级数。
- 洗衣 10 load × 3 级：无流水 30h，流水 3+9 = **12h**；5 级 10 指令：50 vs **14** 周期。
- CPI>1 的原因：填充（可忽略）、**stall**（数据依赖，插 bubble）、**flush**（控制依赖，取错指令变 bubble）。
- 控制依赖：~20% 指令是分支/跳转，>50% taken。ALU 级（第3级）resolve → 每次取错罚 2 周期：CPI = 1 + 0.1×2 = 1.2。

例 25% 是 taken 分支，10 级流水第 6 级算出目标：CPI = 1 + 0.25×5 = **2.25**（级数 10 无关）。

依赖 (Dependence) vs 冒险 (Hazard)

类型	别名	含义
RAW	flow / **true**	后读前写的值
WAR	anti / false / name	后写不能早于前读
WAW	output / false / name	写顺序要保持（最后值正确）
RAR	—	**不是**依赖

- Dependence 只是**程序**属性；Hazard = 依赖在**特定流水线**中会导致错误结果。

- 经典 5 级中 WAR/WAW **永远不是 hazard**（按序读、按序写）；RAW 当生产者与消费者之间 **< 3 条**指令时是 hazard（D 读、W 写）。
- 3 级流水（F/D | 读+ALU | MEM+W）：紧邻的下一条是 hazard，隔一条就不是。
- 处理：控制 hazard → **flush**；数据 hazard → **stall** 或 **forward**（优先 forward）；load-use（MEM 末才有值，下一条 ALU 要用）→ **必须 stall 1** 再 forward。
- 级数选择：级多 → hazard 罚更多（CPI↑）但周期短；纯性能最优 ~30–40 级；加上功耗（latch 能耗、频率）→ 现代 **10–15 级**。

4 • Branch Prediction

- 取指时只知道 **PC**（还没解码！），要猜下一 PC：是不是分支？taken 吗？目标？ → 可归结为"**是不是 taken branch**"。
- **不预测**（等确定再取）：非分支也要 2 周期（D 后才知道不是分支），分支 3 周期。**Predict not-taken**（PC+4）：非分支 1，分支 1 或 3 → 永远不差于不预测。
- 分支在错误路径的"影子"里被误预测 **不额外罚**（它自己会被 flush）。

例 resolve 在第 3 级（罚2）vs 第 10 级（罚9），20% 分支：50% 准确 → 1.2 / 1.9；90% → 1.04 / 1.18。不预测 vs 完美预测的 3 指令循环：7 vs 3 周期（2.33×）。
例 Not-taken 准确率 ≈ 80% + 0.4×20% = 88%，misp=12%：5级 1.24；14级(罚10) 2.2；4-wide 14级：0.25+0.12×10 = 1.45。99% 预测器：1.02 / 1.1 / 0.35 → 越深、越宽越需要好预测器。
例 P4 Prescott：CPI 0.5，1% 误判，罚 30：ideal = 0.5 − 0.2×0.01×30 = 0.44；2% → 0.44+0.12 = **0.56**。

BTB / BHT

- **BTB**（Branch Target Buffer）：PC 索引 → 预测的下一 PC/目标。在执行时比较，错则更新。必须单周期 → 小；只需覆盖"最近会执行"的指令。
- 索引用 PC **最低的、会变化的**位（不是高位：同循环的指令会冲突）。固定 4B 对齐指令 → **丢掉最低 2 位**。
- **BHT**：PC 索引，每项很小（1-bit 或 2BC）→ 可以很大。BHT 说 taken 才查 BTB；只把 taken 分支放进 BTB。

> ▶ **做题步骤** 1024 项 BTB，PC = 0x0000AB0C：去掉低 2 位 → 取 bits[11:2] = 0x2C3。BHT 16 项、指令从 0xC000 起：entry = (PC>>2) mod 16（0,1,2...）。BHT 每次取指都访问；BTB 只在预测 taken 时访问（完美 BHT 下 = taken 次数）。

1-bit / 2-bit 预测器

- 1-bit = 上次结果。问题：每个 anomaly **误判 2 次**（异常本身 + 回到常态那次）；短循环每次进入+退出各错 1 次；T/N 次数相近的交替模式很差。100 次循环例：600+ 次执行只错 2 次。
- **2BC**：00 强N、01 弱N、10 弱T、11 强T；**高位 = 预测**，低位 = hysteresis/conviction。T 加 1、N 减 1，饱和。单次 anomaly 只错 1 次；行为翻转要错 2 次。
- 初始状态：从弱态开始猜错少错一次，但交替模式下可能 100% 错；实际影响很小 → 00 最易初始化。
- **任何预测器都有 100% 误判序列**（例：00 起 T,T,N,T,N...）。3-bit 只在 anomaly 成串时有用 → 一般 2-bit 足够。

History-based

- 每项：N-bit 本地历史 + 2^N 个 2BC；用历史选计数器。**N-bit 历史能预测长度 ≤ N+1 的模式。**
- 1-bit 历史 + 2×2BC：NT 交替 OK；NNT 每轮错 1（100 轮 = **100** 次误判）。2-bit 历史：NNT 用 3/4 个计数器（11 从不出现）；NT 只用 2 个。
- 浪费：只用 ~N+1 个计数器。8 次迭代循环需要 8-bit 历史、256 个计数器只用 9 个。

N	每项 bits	×1024 项	NNNT?	NT 用几个 2BC
1	1+4=5	5,120	✗	2
4	4+32=36	36,864	✓	2 / 16
8	8+512=520	532,480	✓	2 / 256
16	16+131072	134,234,112	✓	2 / 65536

Shared counters: PShare / GShare

- **PShare**：PC 低位索引 PHT（每分支**私有**历史，只存历史位）→ 历史 XOR PC → 索引共享的 2BC 表（每项 2 bit）。可能冲突但表大时很少。成本例：2^{11}×11 + 2^{11}×2 ≈ **26 Kbit**。always-taken 分支只用 1 个计数器。
- **GShare**：一个**全局**历史（所有分支结果移入）XOR PC。擅长**相关分支**（if(shape==square)... if(shape!=square)）。
- PShare 擅长自身模式（奇偶、短循环）；GShare 需要更长历史才能做同样的事。例：for 循环里奇偶分支：PShare 历史 1 bit；GShare 需 **3 bit**（夹着另 2 个分支）。
- **Tournament**：两个好预测器 + meta-predictor（每 PC 一个 2BC，选用哪个）；两者都更新；meta 只在**一对一错**时计数（GShare 对→减，PShare 对→加）。
- **Hierarchical**：便宜预测器 + 昂贵好预测器；只有便宜的不行时才把分支放进/更新好的（带 tag）→ 好的可以项少但很强。通常优于 tournament。Pentium M：2BC → local（有 tag 用它）→ global（有 tag 优先）；2BC 错就插入 local，local 错就插入 global。
- 组合题模板：**Hierarchical(2BC , Tournament(PShare, GShare))**。

RAS (Return Address Stack)

- 函数 return 方向容易（总 taken），目标难（多处调用）→ BTB 每次都错。RAS：call 时 push 返回地址，ret 时 pop。
- RAS 满了：**wrap-around**（覆盖最旧）优于 don't push —— 小函数调用多，牺牲大函数的最终返回。它只是预测器，错了也只是 mispredict。
- 怎么在取指时知道是 RET？用一个简单的 1-bit 预测器，或 **predecoding**（I-cache 里每条指令多存位：是否 RET/分支/长度）。

5 • Predication

- 预测对 → 0 损失，错 → 丢弃 ~50 条（12 级×4-wide≈48）。Predication：两条路都做，必定浪费一半。
- **循环**：predict（predicate 会指数分叉）；**call/ret**：predict；**大 if-then-else**（各 100 条）：predict（浪费 100 > 误判罚 50）；**小 if-then-else**（各 5 条）：浪费 5 vs 0.1×50=5 → 准确率 <90% 时 predicate 更好。
- **If-conversion**：编译器把两条路都算出来再选。如果用分支来选 = 没用。需要条件移动指令。
- **MIPS**：`MOVZ Rd,Rs,Rt`：if Rt==0 then Rd←Rs；`MOVN`：if Rt≠0。x86：`CMOVcc`（看 flags）。

例 `BEQZ R1,Else; ADDI R2,R2,1; B End; Else: ADDI R3,R3,1; End:` → `ADDI R4,R2,1; ADDI R5,R3,1; MOVN R2,R4,R1; MOVZ R3,R5,R1`（4 条无分支）。

> ▶ **做题步骤** 临界点：if-converted 4 条 vs 原代码平均 2.5 条 + p×30 → p = 1.5/30 = 5% → 准确率 **< 95%** 时 if-convert 更好。（另例：80% 准、罚 40：2.5+0.2×40 = 10.5 ≫ 4。）

- CMOV 方式缺点：需编译器支持（旧代码不受益）、需要**更多寄存器**、**额外选择指令**。
- **Full predication**（Itanium）：每条指令带 qualifying predicate；41-bit 指令，低 6 bit 指定 64 个 1-bit predicate 寄存器之一。`cmp.eq p1,p2 = R1,0` 设互补谓词，然后 `(p2) ADDI R2,R2,1`、`(p1) ADDI R3,R3,1` → 不需额外寄存器和 MOV。

例 full-pred 3 条、CPI 0.5 = 1.5 周期；分支版平均 2.5 条 = 1.25 周期 + p×10 → p = 2.5% → 准确率 < **97.5%** 时 if-convert。

6 • ILP

- 同一周期执行所有指令不行：RAW 依赖要求值先产生（forwarding 不能"时间倒流"）。
- WAW 例：I2 因 RAW 延后，I5 先写 R4 → 最终值错 → 需按序写或**重命名**。
- False/name 依赖（WAR, WAW）只因复用同一寄存器名 → 用 **register renaming** 消除（方案一：保留所有版本 = 太复杂）。
- **RAT**（Register Allocation/Alias Table）：架构寄存器 → 当前物理寄存器。

> ▶ **做题步骤** Renaming：每条指令 ① 先在 RAT 查**源**寄存器 → 物理名；② 再为**目的**分配新物理寄存器并更新 RAT。（顺序反了会等自己的结果！）

ILP 定义：处理器每条指令 1 周期完成、宽度无限、完美同周期分支预测、无结构冒险、重命名 → 只受 **RAW** 限制的 IPC。**ILP 是程序的属性**，与具体处理器无关（"i7 上的 ILP"没有意义）。

▶ **做题步骤** 算 ILP：不必真的重命名 —— **只画 RAW 边**，忽略 WAR/WAW 和控制依赖；每条指令周期 = 1 + max(其生产者周期)；ILP = 指令数 / 最大周期（别除反）。分支照样等它的 RAW，但它后面的指令不等它。

- IPC 还受：issue 宽度、in-order、功能单元数量（如只有 1 个 ALU）。例：ILP 2.5，2-issue OoO 1 MUL + 2 ALU → 2.5；只有 1 个 ALU → 1.25。3-issue in-order，ILP 3 → IPC 2（in-order 停在第一个依赖处）。
- Narrow in-order：受 issue 宽度限制；wide in-order：受 in-order 限制 → 需要 **wide + OoO + renaming**。

7 • Instruction Scheduling (Tomasulo)

- IBM 360（40+ 年）；当时只用于 FP 指令、窗口小、异常处理弱；现代 OoO 基本同构。
- 结构：**IQ**（按序）→ 每类单元各自的 **RS**（reservation stations）；**RAT**；**RF**；结果经 **CDB**（common data bus）广播给 RF 和所有 RS；Load/Store：地址生成后进 load/store buffer，**按序**访存（原始 Tomasulo 不重排 L/S）。
- 三步：**Issue**（IQ→RS，按序）→ **Dispatch**（RS→执行单元，乱序）→ **Write result / Broadcast**（乱序）。

Issue

- 取 IQ 中下一条（**必须按程序序**，否则重命名错）；找**正确类型**的空闲 RS，没有 → 本周期不 issue（后面的也不能 issue）。
- 源操作数：RAT 空 → 从 RF 读值；RAT 有 tag → 记下等待的 RS 名。然后 RAT[dest] ← 本 RS 名。

Dispatch

- 操作数全部就绪的指令可派发；每单元每周期 1 条。多条就绪时启发式：**oldest first**（常用）、most dependents first（难实现）、random（正确性 OK）。
- 就绪了却没派发的合理原因：上周期才 issue；单元被别的占了。**不会**因为"有更老的没派发"而等（那就是 in-order 了）。

Write Result

- CDB 上放 (tag, value) → 所有 RS 比对两个操作数 tag 并捕获 → 若 RAT[某 reg]==tag：写 RF 并把 RAT 清空（指回 RF）→ 释放 RS。
- 多个同时完成、只有 1 条 CDB：通常给**慢单元**（mul/div）优先（等它的指令多）。多条 CDB → 每个 RS 比较器数 × 总线数。

⚠ **易错 Stale broadcast**：tag 已经不在 RAT 里（dest 被后来的指令重命名了）→ 只给 RS 捕获，**不写 RF、不改 RAT**。做题别顺手把 RAT 项清掉！

同周期规则（默认，题目会另行规定）：issue→dispatch 同周期：通常**不行**；capture→dispatch 同周期：通常不行；同一 RAT 项 issue 与 broadcast 同周期都要写：**以 issue 的为准**（更新的指令）。
Tomasulo：issue **按序**；dispatch、write result **乱序**。

▶ **做题步骤** Timing 表：
① Issue(i) = Issue(i−1)+1，但若无空闲 RS → 等到某 RS 释放的**下一个**周期（RS 释放当周期不能给新指令；Tomasulo 中 RS 在 **broadcast** 时才释放）。
② Exec 开始 = max(Issue+1, 最后一个操作数 broadcast 周期 +1)（若允许 capture&dispatch 同周期，则看具体规则）。
③ Write = Exec 开始 + latency（长例中：exec 2 周期 → 第 3 个周期写）；CDB 冲突时按优先级推迟。
④ 非流水的单元：下一条要等上一条执行完。在旁边写"因 RS/因 R2 延迟"的注释。

8 • ReOrder Buffer (ROB)

- Tomasulo 的问题：① **不精确异常**（DIV 除零在周期 40 才发现，后面指令已写寄存器）；② **分支误预测恢复**（错误路径已写 R3）；③ **phantom exception**（不该执行的指令触发异常）。
- 解决：可以乱序执行、乱序广播，但**按序写寄存器（commit）**。
- ROB 项：**value**、**done** 位、**dest reg**；按程序序；issue 指针（新指令放这）与 commit 指针（最老）。

步骤	Tomasulo	+ROB
Issue	需空 RS；RAT→RS#	需空 RS **且** 下一个 ROB 项；RAT→**ROB#**
Dispatch	RS 保留	派发即**释放 RS**（名字改由 ROB 承担）
Broadcast	写 RF、更新 RAT、释放 RS	tag=ROB#；RS 捕获；写入 ROB value、done=1；**不动 RF/RAT**
Commit	—	按序：head 项 done 才提交；值**无条件**写 RF；只有 RAT[dest]==本 ROB# 时才把 RAT 指回 RF；释放 ROB 项

- ROB 的作用：记住程序序；暂存结果直到 commit；作为结果的名字/tag。**不是**：存源操作数（RS 的事）、决定哪个单元（RS 的事；ROB 是统一的）。
- 同样 RS 数量下，无 ROB 更容易因没 RS 而卡 issue（RS 占用到 broadcast）。
- **分支误预测恢复**：分支执行时在 ROB 里标 mispredicted；等它 **commit** 时：此时 RF 正好是分支点的正确状态 → 清空 ROB（issue 指针 = commit 指针）、RAT 全部指回 RF、清空 RS 与执行单元，然后从正确 PC 取指。
- **异常**：把异常当作"结果"记在 ROB 中；到 commit 时：前面都已提交、它和后面都未提交 → flush，跳 handler（精确）。phantom 异常永远到不了 commit。
- 程序员只看 commit（"officially executed"）。
- 为什么 commit 时总写 RF（即使会被覆盖）：保证任何时刻 RF = commit 点的精确状态，随时可 flush。

▶ **做题步骤** ROB timing：Issue（按序 + RS + ROB 项）→ Exec = max(Issue+1, 操作数就绪) → Write = Exec+lat → **Commit = max(Write+1, 上一条 Commit+1)**（若可多发 commit 则按宽度）。**注意题目规定 RS 在 dispatch 还是 broadcast 释放**。例：DIV 40 周期 → 后面所有指令 commit 都排在 43 之后（44,45,...）。

- **Unified RS**：所有 RS 一个池，提高利用率（某类满时不会卡 issue）；代价：dispatch 选择逻辑更复杂。
- **Superscalar**：fetch、decode、issue（连续多条，按序，前一条卡住后面都卡）、dispatch、broadcast（每 RS 比较器 ∝ 总线数）、commit（前面都提交才能提交）都要 >1/周期；**最窄的一级决定 IPC**。
- **术语**：Issue = allocate = (有人叫) dispatch；Dispatch = execute = (有人叫) issue；Commit = retire = complete = graduate。
- OoO 处理器中**按序**：fetch、decode、issue、commit（及之后）；**乱序**：dispatch、execute、broadcast。

9 • Memory Ordering (LSQ)

- 寄存器依赖已由 renaming+Tomasulo 处理；但 load/store 可通过**内存地址**产生依赖（RAW: SW→LW 同址；WAR: LW→SW；WAW: SW→SW）。
- Store 在 **commit** 时才写 cache/内存（之前随时可能被取消，撤销内存写极难）。
- **LSQ**：像 ROB 一样按序、commit 时移除，只放 L/S；字段：L/S 位、地址、值、完成位。
- Load 算出地址后：向前（更老的）找**最近的**同址 store → 有：**store-to-load forwarding**（不访存）；无：访问 cache。

前面有 store 地址未知时：
① 所有 L/S 按序执行（最慢，正确）；② 等所有更老 store 的**地址**已知（地址匹配但值未出→等值）；③ **Go anyway**（现代处理器）：不匹配就直接去内存；之后 store 地址解析时检查后面已执行的同址 load → 若有，**重做该 load 及其后续指令**。另有预测哪些 load 常冲突的方案。

例 3 对 LW/SW，load 都 miss（40 周期）：乱序 44 周期；按序：LW 1→41, SW 42, LW 43→83, SW 84, LW 85→125, SW **126**。

- Issue L/S 需要 **ROB 项 + LSQ 项**（LSQ 充当 RS）。执行 = 算地址 + (load) 取值。
- Load 拿到值后要 **broadcast**；store **不 broadcast、没有 ROB 结果**。Load 从前面 store 拿值时来源是 **LSQ 项**（不是 CDB/RS/ROB）。
- Commit：load = 释放 ROB 和 LSQ 项；store 另外把写发给 cache。cache/内存内容永远 = commit 点状态。

10 • Compiler ILP

- 编译器可以：① 提高程序本身 ILP（打断依赖链）；② 让独立指令靠近（硬件窗口/ROB 有限）。
- **Tree height reduction**：`((R2+R3)+R4)+R5` → `(R2+R3)+(R4+R5)`；依赖**结合律**，不是所有运算都能做（浮点要小心）。

例 `R1+R2–R3+R4–R5+R6–R7`（6 条串行，ILP 1）→ `R10=R1+R2; R11=R4+R6; R10=R10+R11; R11=R3+R5; R11=R11+R7; R10=R10–R11` → 3 周期，ILP **2**。

- **Instruction scheduling**：把不相关的指令挪进 stall 槽。挪动 `ADDI R1,R1,4` 到 load 后面 → 后面的 `SW 0(R1)` 要改成 `SW –4(R1)`。挪 load 到前面可能要**换寄存器**（R1→R10）避免覆盖。
- 例：LW 2 周期、ADD/SW 1：`LW,ADD,SW,LW,ADD,SW` 8 周期 → 调度+重命名后 6 周期。
- If-conversion 后代码无分支 → 可以跨原 then/else 边界自由调度。循环不能 if-convert（每次迭代新谓词）→ 用 **loop unrolling**。

> ⚠ **易错 Unroll once = 每次新迭代做 2 次原迭代**；unroll twice = 3 次。复制工作体、调整第二份的 offset（如 `–4(R1)`）、指针步长改为 2×（–8）、只保留一个分支。

- Unrolling 好处：① **IC↓**（消除 loop overhead：5×1000=5000 → 8×500=4000）；② 调度空间更大 → **CPI↓**。4-issue in-order 例：原 CPI 3/5 → 调度后 2/5；unroll 1 次未调度 5/8，调度后 **3/8**。
- 缺点：**code bloat**；迭代次数未知 / 不是倍数（课程不深入）。
- **Function inlining**：消除 call/ret 及参数准备开销（IC↓），并让函数体可与调用处一起调度（CPI↓）。缺点 code bloat → 只内联**小函数**。例：10 周期 → 内联后 7 周期。
- 其它：**software pipelining**（把循环当流水线，不同迭代的不同阶段并行，代码不膨胀）；**trace scheduling**（"if-conversion on steroids"：常走路径拼成一块调度，偏离时跳出并执行补偿代码）。

11 • VLIW

	OoO superscalar	In-order superscalar	VLIW
每周期	≤N 条	≤N 条	1 条大指令（= N 个 op）
找独立指令	在 ≫N 的窗口里找	只看接下来 N 条	不找，编译器给
硬件成本	高	较低	最低
编译器	有帮助	很需要	**完全依赖**

- 代码大小：32-bit 指令程序 4000B → 128-bit/4-op VLIW：**4000 ~ 16000B**（全依赖时每条只放 1 op + 3 NOP）。
- 优点：编译器有充足时间排程；硬件简单；能效高；擅长**循环与规则代码**（数组、矩阵）。
- 缺点：延迟不确定（cache miss）；不规则代码（决策多、指针多：AI、链表）；**code bloat**（NOP）。
- ISA 特征：普通 opcode、**full predication**、**大量寄存器**（调度要用）、**branch hints**、**指令压缩**（stop bits 减少 NOP）。
- 向后兼容：每周期取 2 条旧 64-bit VLIW 指令 → **不是真 VLIW**（两条之间可能有依赖，要硬件检查）。
- 例子：Itanium（硬件反而极复杂，不规则代码仍差）；**DSP**（规则浮点循环，性能与能效极好）。最适合"把很多数相加"，不适合迷宫寻路、链表计数。

12 • Cache Review

- **Temporal locality**：同一地址很快再访问（循环变量 j、sum）；**Spatial**：附近地址很快被访问（数组元素 arr[i]；arr 本身无时间局部性）。New Year's Eve 是反例。
- Cache = 小而快、放在处理器内（借书回家，而不是每次去图书馆 / 在家建图书馆）。
- AMAT = HT + MR×MP；另一写法 `(1–MR)×HT + MR×(HT+MP)`。设计在 hit time（小快）与 miss rate（大/聪明）之间平衡。好 cache：HT ≪ miss time，miss time ≈ MP，hit rate ≈ 1。
- L1：**16–64KB**，~90% hit，**1–3 周期**。Block **32–128B**：≥ 最大单次访问、利用空间局部性；太大（1KB）浪费容量。必须 **2 的幂**（48B 不行）且**对齐**（block 起始地址是块大小的倍数）。
- Line = cache 中的槽；block = 放进去的内存内容（大小相同）。

例 32KB、64B 块，N 个无空间局部性的变量：每个独占一块 → N ≤ 32K/64 = **512**。

- **Tag**：不含 offset 位；至少含 1 位 block number；DM/SA 中不含 index 位（同一 set 的块 index 位相同）。
- **Valid bit**：上电全部置 0；hit = tag 匹配 **且** valid=1（tag/data 初值随意）。
- 类型：**Direct-mapped**（1-way，快、便宜、省能，但**冲突**）、**N-way set-assoc**（"2-way"= 每 set 2 块，不是 2 个 set）、**Fully assoc**（无 index 位）。

> ▶ **做题步骤** 地址拆分：**先 offset**（$\log_2$ block）→ 再 index（$\log_2$ #sets，#sets = size/block/ways）→ 剩余为 tag。冲突 = index 相同且 block number 不同（同一 block 不叫冲突）。
> 例：16KB DM，256B 块：offset 8 bit（低 2 hex 位）、64 sets → index 6 bit；0x12345678 与 0x11115678、0x12341666 冲突，与 0x12345677 同块。
> 例：256B、32B 块、2-way、32-bit 地址：offset [4:0]，index [6:5]，tag [31:7]。
> 例：4KB 4-way 64B，64-bit：offset 6，16 sets → index 4，tag 54；每行额外 valid1+dirty1+tag54+LRU2 = 58 bit。

替换策略

- Random、FIFO、**LRU**（好但贵）、**NMRU**（记录 MRU，其余随机）。

> ▶ **做题步骤** LRU 计数器（N-way，值 0…N–1 互不相同，0 = LRU）：访问块 X（原值 c）→ X 置 N–1；所有 **> c** 的计数器减 1，< c 的不变。替换计数器 = 0 的块（新块随后置 N–1）。已是 MRU 再访问：不变。每步检查计数器互不相同。

写策略

- Allocate：**write-allocate**（写 miss 也调入，常用）vs no-write-allocate。
- **Write-through**（每次写都写内存）vs **write-back**（替换时才写回，需 **dirty bit**；clean 块替换不写回）。Write-back 通常配 write-allocate。
- 写命中只需把 dirty 置 1（不必先检查）。valid=0 时 dirty/tag 无意义。

例 单行 DM，初始 V=0,D=1,tag=A；R A, R B, W B, R C, R D, W D → 4 次 miss、1 次写回（B），终态 V=1,D=1,tag=D。

13 • Virtual Memory

- 程序视角：巨大的线性地址空间（code、static、heap↑ … 栈↓），每个进程都从 0 开始；硬件视角：实际内存条大小。16 个 4GB 地址空间的进程可以在 4GB/16GB/… 物理内存上跑。
- 物理地址 ↔ 物理位置一一对应。虚拟内存大部分（heap 与 stack 之间）永不访问。
- **Page**（虚拟，对齐，典型 4KB）↔ **Frame**（物理槽）；OS 维护每进程**页表**；共享内存 = 两进程的页映射到同一 frame。放不下的页在**磁盘**上，需先调入。
- 翻译：VA = VPN | page offset；PT[VPN] → PFN；PA = PFN | offset（offset 不变）。

例 2GB 物理、4GB 虚拟、4KB 页：frames = $2^{31}/2^{12} = 2^{19}$；每进程 PT 项 = $2^{32}/2^{12} = 2^{20}$。

例 16-bit VA，4 项 PT → VPN = 高 2 bit，offset = 14 bit；20-bit PA = PFN(6 bit) | 14-bit offset。

- Flat PT 项 ≈ 物理地址大小（PFN + 保护/存在位）。4GB/4KB×4B = **4MB/进程**（即使只用一点）。64-bit 时根本放不下。

例 8B PTE、4KB 页、2 个进程、32-bit VA（不管它们各用 1MB/1GB、物理 2GB）：$2 \times 2^{20} \times 8$ = **16MB**。

多级页表

- VPN 拆成 outer | inner；outer 表项指向 inner 表；整段未用 → 对应 inner 表**不存在**。全部 inner 表总大小 = flat 大小（+ outer），节省来自省掉的 inner 表。

例 32-bit、4KB、10|10|12、8B PTE，只用最低的十几页（VPN 0x00000–0x00010）和最高的 16 页（VPN 0xFFFF0–0xFFFFF）：flat = 2^{20}×8 = 8MB；两级 = outer 8KB + 2 个 inner×8KB = **24KB**。

例 64-bit、64KB 页、8B PTE、用 0~4GB：flat = 2^{48}×8 = 2^{51}B；4 级（48 = 4×12）：最内层需 2^{16} 页/2^{12} = 16 张 + 上面 3 级各 1 张 = 19 张 ×

(2^{12}×8 = 32KB) = **608KB**。

- 页大小：大页 → 页表小；但 **internal fragmentation** 大。折中：几 KB ~ 几 MB；x86 4KB（当年更怕浪费内存）。
- 代价：PT 在内存中 → 每次 load/store 先做 n 级 PT 访问。

例 1 周期算地址，cache hit 1，内存 10，hit 90%，3 级 PT：PTE 不可缓存 → 1+3×10+1+0.1×10 = **33**；PTE 可缓存 → 1 + 3×(1+0.1×10) + (1+0.1×10) = **9**（6 周期花在翻译）。

TLB

- 只缓存**最终**翻译（多级也只 1 次访问）；1 项覆盖整页 → 很小很快。16KB 数据只需 4 项。
- Miss 处理：**软件**（OS 填 TLB，PT 可以是任意结构：树、哈希；嵌入式）vs **硬件** page walker（PT 必须是 flat/多级格式；更快；x86）。
- 组织：**全相联/高相联**；64–512 项；不够 → 两级 TLB（L1 单周期；L2 几千项、数周期）。
- 所需项数：32KB cache/64B 块、4KB 页 → 介于 8（cache/页）与 512（#块）之间 → 选 "8~512"。

例 1MB 数组逐字节扫 10 遍，4KB 页 → 256 页；L1 TLB 128 项、L2 1024 项（DM，初始空）：L1 miss = 10×256，hit = 10×4095×256；L2 miss = 256，hit = 9×256。

14 • Advanced Caches

三类方法：降 **hit time** / 降 **miss rate** / 降 **miss penalty**。

降 Hit Time

- 更小/更低相联：HT↓ 但 MR↑，要平衡。
- **Pipelined cache**：stage1 读 tag+valid；stage2 判 hit、开始读 data；stage3 读完 data。L1 2–3 周期时常流水化（hit time 包括等前一访问的时间）。
- **PIPT**：先 TLB 再 cache，hit latency = TLB + cache。
- **VIVT**：用 VA 直接访问；hit 时不需翻译。问题：仍需 TLB 的**权限位**；**context switch 要 flush**；**aliasing**（mmap 两个 VA → 同一 PA，可落在不同 set，写 A 读 B 看不到）。
- **VIPT**：VA 的 index 位查 set，同时查 TLB；用 PA 比 tag。HT = max(cache, TLB) ≈ cache；**不需 flush**；若 index 位全来自 page offset → **无 aliasing**。

VIPT 最大无 aliasing 容量 = associativity × page size。例：4-way、16B 块、8KB 页：offset 4 + index 9 = 13 → 2^9×16×4 = **32KB**。4KB 页、32B 块 → index 最多 7 bit（128 sets）。Intel：P4 4-way 16KB；Core2~Haswell 8-way 32KB；Skylake(传闻) 16-way 64KB。增大只能靠提高相联度。

- **Way prediction**：先猜一路（像小 DM cache），猜错再查整个 set → miss rate 同 SA，hit time 多数像 DM。直接映射不能用。

例 32KB 8-way：90%、HT 2、MP 20 → AMAT 4。4KB DM：70%、HT 1 → 1+0.3×20 = 7。Way-pred：0.7×1 + 0.3×2 = 1.3，+0.1×20 → **3.3**。

- 替换与 HT：Random 命中时无需更新（快但 MR 差）；**LRU** 每次命中可能改 N 个计数器（即使不改也要比较）→ 慢、耗电。
- **NMRU**：每 set 1 个 MRU 指针（$\log_2$N bit，比 LRU 少 N 倍状态）；只保护 MRU，其余不知顺序。
- **PLRU**（pseudo-LRU）：每行 1 bit；访问置 1；替换选 0 位的；最后一个 0 被置 1 时把**其它**清 0。介于 NMRU（1 个 1）与 LRU（只剩 1 个 0）之间；命中只需置位。

例 4 行全相联 NMRU：A A B A C A D A E A A A A B → 前 4 个不同块都放空行；E 替换 B/C/D 之一 → 最后 B 可能命中也可能 miss → 最少 **5**、最多 **6** 次 miss。

降 Miss Rate

3C：**Compulsory**（首次访问，无限大 cache 也有）；**Capacity**（同容量**全相联**也会 miss）；**Conflict**（同容量全相联就不会 miss，因相联度不足）。更大 cache → capacity↓；更高相联/更好替换 → conflict↓（但都影响 HT）。

- **更大 block**：空间局部性好时 MR↓，差时（垃圾占容量）MR↑。小 cache 拐点早（4KB ~64B），大 cache 拐点晚（256KB ~256B）。局部性好时减少 compulsory（几乎必然）、capacity（是）、conflict（可能）。
- **Prefetching**：猜对 → 消除 miss；猜错 → **cache pollution**（还可能制造新 miss）。
- Prefetch 指令：`prefetch(a[i+pdist])`；太小 → too late（仍等，但等得少）；太大 → too early（用前被踢）；且随硬件代际变化。

例 a[i] += b[i][j]，8B 元素，16KB 全相联 LRU，内层 10 周期/次，MP 200：b 的 pdist = 200/10 = **20**（内层）；a 预取 **i+1**（外层一次 10000 周期 > 200；每次外层只带入 8000B < 16KB 不会被踢；i+20 会被踢 = premature）。

- 硬件预取：**stream buffer**（顺序下几块）、**stride**（固定间隔 d）、**correlating**（记录 A 后是 B → 适合重复的链表遍历）。
- **Loop interchange**：C 数组行优先 → 内层循环应遍历最后一维（`a[i][j]` 内层 j）；编译器须证明迭代间无依赖才能交换。

降 Miss Penalty

- **Blocking** cache：一次一个 miss。**Non-blocking**：**hit-under-miss**、**miss-under-miss** → 多个 miss 重叠 = **MLP**（memory-level parallelism）。
- **MSHR**：miss 时查 MSHR：不匹配 → 真 miss，分配新 MSHR，记下要唤醒的指令，发请求；匹配（同块已在途）→ **half-miss**，只把指令加到该 MSHR，不再发请求。数据回来 → 唤醒全部、释放。2 个就有大收益，16–32 更好。
- 不受益：总命中的程序；每 1000 条才 miss 一次（ROB 先满，看不到下一个 miss）。每 10 条、每 2 条 miss → 受益。
- **多级 cache**：L1 miss → L2；最后一级 = **LLC**（miss 直接去内存）。L1 容量 < L2、L1 延迟 < L2；L2 访问数 **少于** L1；相联度不必相同（L1 通常更低）。

例 16KB：HT 2、90% → 2+0.1×100 = 12；128KB 单独：HT 10、97.5% → 12.5；无 cache：100；L1+L2：L2 local hit 75% → 2 + 0.1×(10 + 0.25×100) = **5.5**。

- **Local** miss rate = 本级 miss / 本级访问；**Global** = 本级 miss / 处理器全部访问；**MPKI** = misses / 1000 条指令。L1 90%：local=global=10%；L2 命中一半 L1 miss：local 50%，global **5%**。L2 local hit rate 看起来低是因为 L1 过滤掉了容易的访问。
- **Inclusion**：默认**既非 inclusion 也非 exclusion**（L1 热块的访问不到 L2，L2 的 LRU 会把它踢掉）。强制 inclusion：L2 加 inclusion bit，不替换仍在 L1 的块。维护 inclusion 时 L1 脏块写回**一定是 L2 hit**；不维护时可能 hit 也可能 miss。

考场提醒：做 Tomasulo/ROB/cache 模拟题时先抄下题目给的规则（latency、RS 何时释放、同周期 issue/capture/dispatch 是否允许、几条 CDB、单元是否流水）；每个延迟写一句原因注释，便于部分给分。